FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

BACHELOR THESIS

Jan Burda

Optimizing 3D Bin Packing Using Evolutionary Algorithms

Department of Theoretical Computer Science and Mathematical Logic

Supervisor of the bachelor thesis: RNDr. Jiří Fink, Ph.D.
Study programme: Informatika

Prague 2026

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date ....

Author's signature

TODO

Title: Optimizing 3D Bin Packing Using Evolutionary Algorithms

Author: Jan Burda

Department: Department of Theoretical Computer Science and Mathematical Logic

Supervisor: RNDr. Jiří Fink, Ph.D., Department of Theoretical Computer Science and Mathematical Logic

Abstract: TODO!!!!

Keywords: 3D Bin Packing, Evolutionary Algorithms, Genetic Algorithms

Contents

Introduction **7**

1 Terminology and Problem Description **9**
1.1 Coordinates . . . 9
1.2 Sizes . . . 9
1.3 Region . . . 9
1.4 Rotation . . . 10
1.5 Relationships Between *Regions* . . . 10
1.6 Problem Description . . . 11
1.7 Fitness Function . . . 12
1.8 Data Representation . . . 13
1.8.1 Input . . . 13
1.8.2 Output . . . 13
1.8.3 Datasets . . . 13

2 The Packing Algorithm **15**
2.1 Terminology . . . 15
2.1.1 Pending box . . . 15
2.1.2 Placement Heuristic . . . 16
2.1.3 Opened Container . . . 16
2.1.4 Placed Box . . . 16
2.1.5 Packing Rules . . . 16
2.1.6 Packing Rules Fitness . . . 17
2.2 Empty Maximal Regions . . . 17
2.2.1 Update of a Single Empty Maximal Region . . . 18
2.2.2 Update of an Empty Maximal Regions List . . . 19
2.3 Heuristics . . . 21
2.3.1 Order Heuristics . . . 21
2.3.2 Placement Heuristics . . . 22
2.4 The Box Packing Algorithm . . . 23

3 Evolution **24**
3.1 Representation of an Individual . . . 24
3.2 Fitness Evaluation . . . 24
3.3 Elitist Genetic Algorithm . . . 24
3.3.1 Selection of Parents . . . 24
3.3.2 Uniform Crossover . . . 24
3.3.3 New Population Construction . . . 25
3.3.4 Mutation . . . 25
3.3.5 Memetic . . . 25

4 Experiments **27**
4.1 Comparison of Placement Heuristics . . . 27
4.2 Box Ordering Strategies . . . 28
4.3 Impact of Mutation and Memetic . . . 28

4.4 Influence of Box Weights 30

Conclusion **31**

Bibliography **32**

A User Documentation **33**

B Programming Documentation **34**

Introduction

The nature of today's world is greatly influenced by globalization, which not only connects economies and societies across continents, but also has a profound impact on cultures [1]. One of the main drivers of globalization is the transportation of goods. In order to transport goods effectively, containers are essential, as they allow for the easy movement of goods using various methods of transportation. They are used in trucks, ships, trains, and even airplanes, facilitating a smooth transition of goods when changing the mode of transport. The use of containers has tremendously reduced handling times and costs, making the global supply chain much more efficient [2]. Since the cost of shipping is typically calculated based on the number of containers, their volume, and their weight, any unused space consequently results in increased expenses. In large-scale logistics, where massive quantities of containers are shipped daily, even small improvements in packing efficiency can lead to significant cost savings as well as reduction in emissions. For example, in 2018, containers imported into the United States were, on average, loaded to only about 65% of their capacity [3].

In the context of computer science and optimization, container packing problems generally revolve around three key types [4]. In 3D Bin Packing Problem, the goal is to pack a set of rectangular boxes into the smallest possible number of containers. For Knapsack Loading Problem, each box has an associated value, and the objective is to select a subset of boxes that fits into a single container while maximizing the sum of their values. The aim of Container Loading Problem is to pack all boxes into a single container, which has infinite length, while minimizing the total volume used. All of these problems are indeed NP-hard [4], so heuristic and approximation algorithms are usually used to find good solutions in a reasonable amount of time.

This thesis focuses on a variant of the 3D Bin Packing Problem in which rectangular boxes not only have given sizes, but also an associated weight, and each container has a maximum load capacity. All orthogonal rotations of the boxes are allowed. Furthermore, every box must be fully supported from below, either by the container floor or by the top surfaces of other boxes, ensuring structural stability of the packing. The primary objective is to minimize the number of containers used while respecting these constraints.

In this thesis, we build upon the genetic algorithm proposed by Gonçalves and Resende [5] and derive several algorithmic variants from it. In particular, we investigate whether its performance can be improved by incorporating mutation operators and memetic local search. Within this framework, we also examine the effect of replacing the box-packing order derived through evolution with several predefined ordering heuristics, as well as the effect of allowing evolution to choose the placement strategy for each box instead of using a single fixed strategy. In addition, these variants are compared with two other optimization approaches: mutation-based hill climbing and the physics-inspired method of simulated annealing [6]. Finally, we investigate how the solutions change for different box density values under a fixed container weight constraint. The algorithms are evaluated according to solution quality, measured by fitness, and computational runtime.

The thesis is structured as follows. Chapter 1 introduces the geometric terminology, formal definition of the packing problem and used datasets. Chapter 2 introduces a packing algorithm that deterministically constructs a complete solution from an individual and evaluates its fitness. Chapter 3 describes the evolutionary algorithms applied to the problem. Chapter 4 reports on the experiments conducted and discusses the results. Finally, two appendices provide additional documentation: Appendix A contains user instructions, and Appendix B includes programming and implementation details.

1 Terminology and Problem Description

In this chapter, we will focus on the formal description of the extended 3D Bin Packing Problem. To make this possible, we first need to introduce the terminology related to the representation of 3D space and the objects within it.

1.1 Coordinates

A *Coordinates* is defined as a triplet (x, y, z), representing a position relative to the origin $(0, 0, 0)$, where:

- x denotes the position along the horizontal axis,
- y denotes the position along the vertical axis,
- z denotes the position along the depth axis.

Each coordinate value must be a non-negative integer:

$$x, y, z \in \mathbb{Z}_{\geq 0}.$$

The origin $(0, 0, 0)$ serves as the reference point from which all coordinates are measured.

1.2 Sizes

A *Sizes* of a rectangular object is defined as a triplet (X, Y, Z), where:

- X denotes the length of the edge along the horizontal axis (X-axis),
- Y denotes the length of the edge along the vertical axis (Y-axis),
- Z denotes the length of the edge along the depth axis (Z-axis).

Each of these values must be a non-negative integer:

$$X, Y, Z \in \mathbb{Z}_{\geq 0}.$$

1.3 Region

A *region* is a rectangular object defined by a pair of coordinates $(\mathit{Start}, \mathit{End})$, where:

- *start* denotes the coordinates of the vertex that is closest to the origin $(0, 0, 0)$,
- *end* coordinates denotes the vertex that is furthest from the origin $(0, 0, 0)$.

It is required that the start coordinate is less than or equal to the end coordinate along each axis:

$$\mathit{Start}_x \leq \mathit{End}_x, \quad \text{and} \quad \mathit{Start}_y \leq \mathit{End}_y, \quad \text{and} \quad \mathit{Start}_z \leq \mathit{End}_z.$$

1.4 Rotation

A *Rotation* of an item is defined by permuting its dimensions (X, Y, Z) according to the orientation, represented by an element of the set:

$$\{XYZ, XZY, YXZ, YZX, ZXY, ZYX\}$$

where each orientation corresponds to a rearrangement of the dimensions:

$$\begin{aligned}
XYZ &\to (X, Y, Z) \\
XZY &\to (X, Z, Y) \\
YXZ &\to (Y, X, Z) \\
YZX &\to (Y, Z, X) \\
ZXY &\to (Z, X, Y) \\
ZYX &\to (Z, Y, X).
\end{aligned}$$

This approach maintains the values as non-negative integers but changes their assignment to axes, effectively rotating the item in discrete 90-degree steps.

1.5 Relationships Between *Regions*

Given two *regions* $A = (A_{\text{start}}, A_{\text{end}})$ and $B = (B_{\text{start}}, B_{\text{end}})$, we define several spatial relationships.

All comparisons between coordinates are interpreted componentwise, for instance, $A_{\text{start}} \leq B_{\text{start}}$ means:

$$A^x_{\text{start}} \leq B^x_{\text{start}} \quad \text{and} \quad A^y_{\text{start}} \leq B^y_{\text{start}} \quad \text{and} \quad A^z_{\text{start}} \leq B^z_{\text{start}}.$$

Subregion

We say that region A is a *subregion* of region B if:

$$A_{\text{start}} \geq B_{\text{start}} \quad \text{and} \quad A_{\text{end}} \leq B_{\text{end}},$$

meaning that A lies entirely within the boundaries of B in all three dimensions. We write $A \subseteq B$.

Overregion

We say that region A is an *overregion* of region B if B is a subregion of A:

$$A_{\text{start}} \leq B_{\text{start}} \quad \text{and} \quad A_{\text{end}} \geq B_{\text{end}}.$$

We write $A \supseteq B$.

Intersection

Two regions A and B *intersect* if there is a non-empty volume that they share. Formally, define open intervals for each dimension:

$$I_A^d = (A_{\text{start}}^d, A_{\text{end}}^d), \quad I_B^d = (B_{\text{start}}^d, B_{\text{end}}^d), \quad \text{for } d \in \{x, y, z\}.$$

The regions intersect if and only if the intersection of intervals is non-empty in all dimensions:

$$A \cap B \neq \emptyset \iff \forall d \in \{x, y, z\} : I_A^d \cap I_B^d \neq \emptyset.$$

This definition ensures that the overlapping region has strictly positive volume.

Volume Function

We define a *volume function* V that returns the volume of *Sizes* or *Regions*:

- given **dimensions** $D = (X, Y, Z)$, the volume is:

$$\mathrm{V}(D) = X \cdot Y \cdot Z,$$

- given a **region** $R = (R_{\text{start}}, R_{\text{end}})$, the volume is defined as:

$$\mathrm{V}(R) = (R_{\text{end}}^x - R_{\text{start}}^x) \cdot (R_{\text{end}}^y - R_{\text{start}}^y) \cdot (R_{\text{end}}^z - R_{\text{start}}^z).$$

The result of V is always a non-negative integer (possibly zero).

1.6 Problem Description

We are given a rectangular *input container* C_in with sizes S and a maximum allowed weight $W_{\max}$, and a set of k rectangular *input boxes*. We call this pair the *packing input*. Each input box $i = 1, \ldots, k$ has sizes s_i and weight w_i. We model the container as a *region* $R_c = (\mathbf{0}, D)$, where $\mathbf{0} = (0, 0, 0)$ denotes the origin, and R determines the *end* coordinate of the container region. Similarly, when a box i is placed inside the container with a valid rotation Rot, it occupies a region $r_i = (p_i, p_i + Rot(d_i))$, where p_i is the start coordinate of the input box within the container. We require that r_i is a subregion of R_c as an effect of constraint C2 below.

Let B_c denote the set of boxes placed in container c. We define the following:

- the total weight of container c is:

$$W_c = \sum_{i \in B_c} w_i,$$

- the extension of the volume function for total volume occupied in container c is:

$$V(c) = \sum_{i \in B_c} \mathrm{V}(r_i).$$

Constraints

We impose the following constraints:

(C1) each box i is placed in exactly one container,

(C2) each box i must lie entirely within its container c:

$$r_i \subseteq R_c,$$

(C3) no two boxes i and j within the same container c may intersect:

$$\forall i \neq j \in B_c : \quad s_i \cap s_j = \emptyset,$$

(C4) the total weight of any container c does not exceed its weight limit:

$$W_c \leq W_{\max}$$

,

(C5) boxes may only be rotated by 90° along any axis,

(C6) every coordinate of a box's base must coincide either with a coordinate of the container floor or with a coordinate on the top surface of another box.

Objective

The goal is to assign all boxes to containers in a way that satisfies all constraints (C1)–(C6), while minimizing the number of containers used N. Compared to the classical 3D bin packing problem, this version introduces two key additional constraints: the maximum weight limit (C4), and the full-support condition (C6) that ensures physical stability of stacked boxes.

1.7 Fitness Function

The objective of the fitness function is to numericaly represent the quality of the solution. The lower the fitness, the better the solution should be. The primary optimization goal is to minimize the number of containers used N. In case two solutions use the same number of containers, the one that utilizes the space more efficiently—i.e., with a higher minimum fill ratio (volume or weight) is considered better. Combining these two requirements gives us the following formula:

let $\mathcal{U}$ be the set of used (i.e., opened and non-empty) containers,

then the fitness function is defined as:

$$\text{fitness} = N + \min_{c \in \mathcal{U}} \left(\max \left(\frac{V(c)}{V(D)}, \frac{W_c}{W_{\max}} \right) \right).$$

1.8 Data Representation

1.8.1 Input

The input consists of a description of the input container and a list of input boxes. The container is defined by its dimensions $D = (X, Y, Z)$ and its maximum weight $W_{\max}$. Each input box i is described by a unique identifier, its dimensions $d_i = (x, y, z)$, and its weight w_i. The input is provided in JSON format, where all numerical values are expressed in the same units.

1.8.2 Output

The output is a JSON object containing an array of containers.
Each container includes:

- its unique identifier,
- total weight,
- total occupied volume,
- a list of packed boxes.

Each box entry includes:

- the box identifier,
- applied rotation,
- the region it occupies.

All numerical values are expressed in the same unit system as the input.

1.8.3 Datasets

For the experimental evaluation, a dataset consisting of 120 problem instances inspired by Martello, Silvano and Pisinger [4] was generated. The dataset is divided into four classes, each containing 30 instances. For every class, 10 instances were generated for each value of $n \in \{50, 100, 150\}$. The container dimensions are identical for all instances and are defined as $X = Y = Z = 100$.

Type	**x**	**y**	**z**
1	$[1, X/2]$	$[Y/2, Y]$	$[Z/2, Z]$
2	$[1, X/2]$	$[1, Y/2]$	$[Z/2, Z]$
3	$[1, X/2]$	$[1, Y/2]$	$[1, Z/2]$
4	$[1, X]$	$[1, Y]$	$[1, Z]$

Table 1.1 Box size limits for each type.

Classes 1–3 represent instances with a dominant geometric structure, while class 4 corresponds to completely general instances.

For classes 1–3, the boxes are generated randomly with a probability of 60 % from the corresponding type and 20 % from each of the remaining two types.

This approach preserves the main structural characteristics of each class while introducing additional variability. The box size limits for each type are shown in Table 1.1. From these intervals, the sizes are selected uniformly.

In addition to the geometric properties, weights were assigned to the boxes due to the presence of a maximum weight constraint. Based on this, three different datasets were constructed. Each of them contains the same 120 problem instances, but the assigned weights differ.

In the original dataset, the boxes do not have weights. To simulate this, the weight of every box is set to 1, while the maximum allowed weight of each container is set to $X \cdot Y \cdot Z$, corresponding to the weight of a fully filled container with unit density. In practice, this means the weight constraint is never restrictive, as any container can accommodate all boxes by weight.

The remaining two datasets, referred to as the *medium* and *heavy* datasets, incorporate density. For each box, density is generated from a normal distribution with a specified mean density μ and standard deviation 0.1μ. The weight of each box is then computed as the product of the density and the box volume.

For the medium dataset, the mean density is set to $\mu = 1.2$, while for the heavy dataset it is set to $\mu = 1.5$.

2 The Packing Algorithm

This chapter describes how we can obtain a deterministic packing solution, i.e. a list of containers and boxes packed within them, from the packing input and the packing rules. The packing rules contain information regarding the rotation of each box, the heuristic that will determine how the box is placed in the container, and the order in which the boxes are packed.

First, the list of input boxes is converted into an ordered list of pending boxes using the packing rules. Each pending box contains the input box together with the information about its rotation and the heuristic that will be used to pack the box.

Once we have the list of pending boxes, they can be packed using the box packing algorithm that takes each pending box and tries to pack it into an open container chosen by the placement heuristic assigned to that particular pending box. In case no fitting container and space are found, a new container is opened. Once a pending box is placed into a container, we call it a placed box. This placed box consists of the input box information, its rotation, the space it occupies within the container, and the identifier of that container.

The solution is then a list of open containers, each with a list of placed boxes within that particular container. Since we can assign fitness to each solution, and the packing rules and packing input determine a unique solution, we can also assign fitness to the packing rules themselves with regard to the packing input.

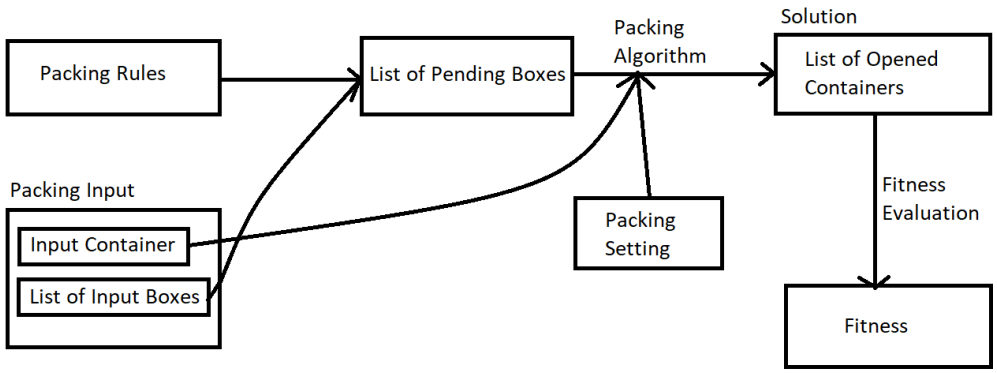

Figure 2.1

2.1 Terminology

2.1.1 Pending box

A *pending box* is represented as a triple (B, Rot, H), where:

- B is the input box,
- Rot is the selected rotation,
- H is the placement heuristic.

The reason each pending box has its own placement heuristic, rather than using a single heuristic for all pending boxes, is to support variability in evolutionary algorithms.

2.1.2 Placement Heuristic

A *placement heuristic* is a function that takes a list of currently opened containers and an input box with its rotation, and returns a selected container along with the region where the box should be placed. Since the input is defined only as a list of opened containers, the heuristic has considerable flexibility. It can take into account various factors when choosing the target container and placement space. For example, if the heuristic had access only to the EMR lists of containers, its decision-making would be restricted to available empty regions. In this more general formulation, however, it can also consider additional attributes such as the weight or volume of each container.

If no valid placement for the box is found (i.e., there is no EMR in any container with sufficient dimensions, or placing the box would exceed the container's maximum weight), then the function returns *null*.

2.1.3 Opened Container

An *opened container* C is represented as a triple (C_{in}, B_c, EMR_c), where:

- C_{in} represents the input container,
- B_c is the set of boxes already placed into the container,
- EMR_c is the set of Empty Maximal Regions within the container.

2.1.4 Placed Box

A *placed box* is represented as (B, C, Reg, Rot), where:

- B is the box,
- C is the container into which the box is placed,
- Reg is the region occupied by the box within the container,
- Rot is the rotation of the box.

2.1.5 Packing Rules

Let $\mathcal{H}$ denote the set of all available placement heuristics, and let $\mathcal{R}$ denote the set of all allowed box rotations. The *packing rules* provide a compact representation of the list of pending boxes and define the order in which they are packed. Formally, they are represented as a triple (o, rot, h), where:

- o is a permutation list of integers from 1 to k. The value at index i specifies the packing order of the box with identifier i.

- rot is a list of length k, where the element at index i represents the rotation assigned to the input box with identifier i, $rot \in \mathcal{R}^k$.
- h is a list of length k, where the element at index i represents the placement heuristic assigned to the input box with identifier i, $h \in \mathcal{H}^k$.

In other words, the packing rules determine the packing order, rotation, and placement heuristic for each input box, which together form the corresponding list of pending boxes.

Vector representation

In order to use the packing rules as an individual for the evolutionary algorithm, they can be represented as a continuous vector of length $3k$ consisting of real values in the interval $[0, 1]$ [5]. This vector is divided into three parts corresponding to the components (o, rot, h).

In this triple, the first k entries are sorted in ascending order to derive the permutation o, the second k entries are interpreted as uniformly distributed indices to select elements from the set of allowed rotations $\mathcal{R}$, and the third k entries are interpreted similarly to choose elements from the set of available heuristics $\mathcal{H}$. This transformation allows evolutionary algorithms to operate on the vector representation while still producing valid discrete packing rules.

If the set $\mathcal{H}$ contains only a single heuristic, the heuristic part becomes redundant, and the vector can be reduced to two parts with a total length of $2k$. A similar situation arises if rotations are not allowed, meaning that no rotation can be applied and the only feasible orientation is the default XYZ. In this case, the rotation part is unnecessary, and the vector can also be shortened to length $2k$. If neither of these parts are needed, the vector can be shortened further to length k. Furthermore, if the packing order is determined directly by a heuristic, the order part can also be omitted, as the order can be derived without relying on the packing vector.

2.1.6 Packing Rules Fitness

Previously, the concept of fitness was defined in the context of evaluating a solution. It is now evident that the packing input together with packing rules uniquely determine a specific solution. This allows fitness to be assigned not only to the solution itself but also to the packing rules with respect to the given packing input. The procedure is as follows: the packing rules are first converted into a list of pending boxes in a specified order, the packing algorithm is then applied to generate the solution, which is subsequently evaluated to determine its fitness.

2.2 Empty Maximal Regions

To keep track of unoccupied regions during the packing process, we assign each opened container a list of *empty maximal regions* (EMR). An empty region is said to be *maximal* if it is not a subregion of any other empty region. In the literature, the more common term is empty maximal spaces (EMS). However, the

term regions is adopted here to ensure consistency with the terminology of this work [7, 5].

Initially, each empty opened container c has a single EMR, which is the region of the container itself, denoted as R_c. As boxes are placed into the container, the set of EMR is updated accordingly.

2.2.1 Update of a Single Empty Maximal Region

Assume we have an Empty Maximal Region:

$$\text{EMR} = (\text{Start}, \text{End}) = ((x_s, y_s, z_s), (x_e, y_e, z_e)),$$

and a region now occupied by a newly placed box:

$$\text{BoxRegion} = ((x_b^s, y_b^s, z_b^s), (x_b^e, y_b^e, z_b^e)).$$

If EMR and BoxRegion do not intersect, the original EMR remains unchanged. If they do intersect, we define their intersection as:

$$\text{IntersectRegion} = ((x_i^s, y_i^s, z_i^s), (x_i^e, y_i^e, z_i^e)) = \mathit{EMR} \cap \mathit{BoxRegion}.$$

In the general case, this intersection splits the original EMR into up to six new EMR subregions, as follows:

- **Left slab (along X)**:

$$((x_s, y_s, z_s), (x_i^s, y_e, z_e))$$

- **Right slab (along X)**:

$$((x_i^e, y_s, z_s), (x_e, y_e, z_e))$$

- **Front slab (along Y)**:

$$((x_s, y_s, z_s), (x_e, y_i^s, z_e))$$

- **Back slab (along Y)**:

$$((x_s, y_i^e, z_s), (x_e, y_e, z_e))$$

- **Bottom slab (along Z)**:

$$((x_s, y_s, z_s), (x_e, y_e, z_i^s))$$

- **Top slab (along Z)**:

$$((x_s, y_s, z_i^e), (x_e, y_e, z_e))$$

Each of these regions is only created if it has positive volume and satisfies the definition of a valid region, i.e., its start coordinates are less than or equal to its end coordinates in all dimensions.

When constraint C6 is applied, we have $z_b^s = z_s$, and thus the bottom slab is never created, reducing the number of new EMR to at most five. Another consequence of constraint C6 is that the top slab must lie directly above the placed box. Therefore, its x and y coordinates are restricted to match the footprint of the box, making it more compact than in the general case:

- **Top slab (along Z)**:

$$((x_i^s, y_i^s, z_i^e), (x_i^e, y_i^e, z_e)).$$

It is worth explicitly mentioning that there is no requirement for the newly generated EMR to be non-overlapping. Overlaps are allowed and common in this representation.

2.2.2 Update of an Empty Maximal Regions List

Since a newly placed box can intersect multiple EMR, it is necessary to update each region in the EMR list using the method described above. Once this is done, several additional steps must be taken [5].

First, we check whether any of the newly created EMR is a subregion of another EMR in the list. Such subregions do not satisfy the definition of maximality and are therefore removed. Second, we attempt to merge adjacent EMR that lie next to each other and start at the same height. Due to constraint C6, each newly generated top slab lies directly above the placed box. However, the constraint still allows a box to rest on top of multiple smaller boxes, as long as its base is fully supported. This means that EMR starting at the same height and sharing a face can potentially be merged into a larger EMR.

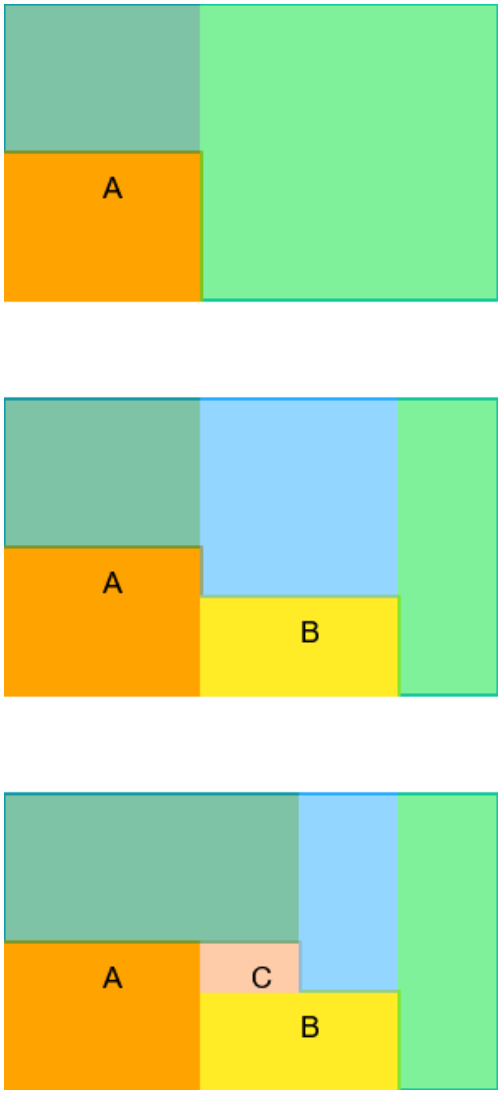

Figure 2.2 Side view of EMR update. Boxes are inserted sequentially in the order A, B, C. The top EMR is always located directly above a box so that the support constraint is satisfied. When two box tops are at the same height, as in the case of A and C, their top EMRs merge into a single EMR.

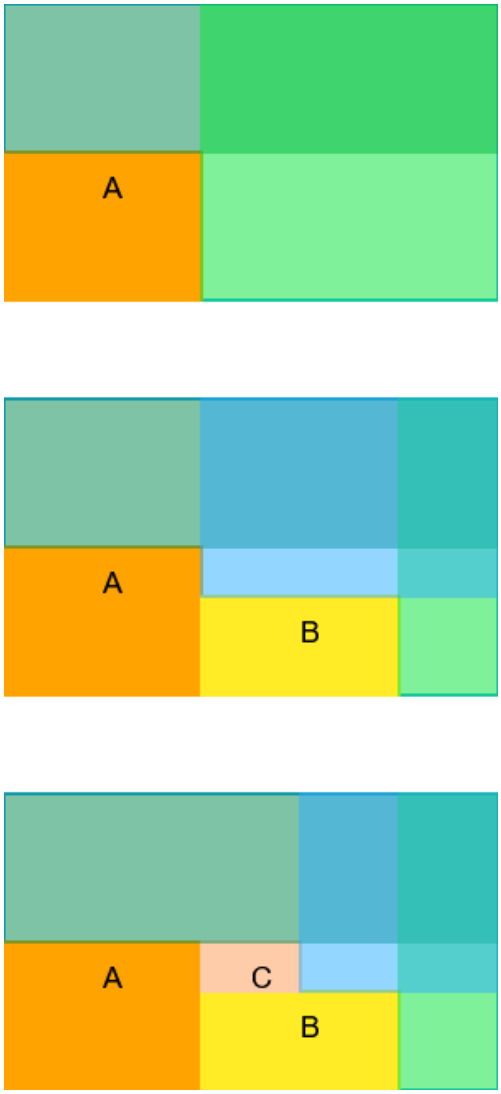

Figure 2.3 Top view of EMR update. Boxes are inserted sequentially in the order A, B, C. Newly added boxes split existing EMR into several smaller regions, and if any of these EMRs is a subregion of another EMR, it disappears. For example, after inserting box C, part of the blue EMR becomes part of the dark green EMR.

2.3 Heuristics

2.3.1 Order Heuristics

If it is not desired for the evolutionary algorithm to determine the order of the boxes,the boxes can instead be pre-sorted using a simple ordering heuristic. In this case, the component of the packing rules responsible for ordering the boxes can be omitted.

The following predefined ordering heuristics are available in the algorithm:

- **High Volume First:** Boxes are sorted by descending volume, placing the largest boxes first.
- **High Area Base First:** Boxes are sorted by descending base area (using rotated sizes), prioritizing those with the largest footprint.

- **High Weight First:** Boxes are sorted by descending weight, useful when container weight is often the limiting factor.

By using one of these heuristics, the packing order is fixed a priori and each box is placed according to the selected heuristic without relying on evolutionary selection for ordering. The evolutionary algorithm still operates, but its role is limited to selecting the rotations and placement heuristics for the boxes.

2.3.2 Placement Heuristics

The following heuristics are used in the experiments.

1. **Best Volume Fit**
   This heuristic selects the EMR with the smallest volume that can still accommodate the box. The goal is to place the box into the tightest possible region in order to reduce fragmentation of the remaining free space.

2. **Max Distance**
   For each feasible EMR, the Euclidean distance between the starting coordinate of the region and the far corner of the container is computed. The heuristic then selects the EMR that maximizes this distance.

3. **Min X**
   The heuristic selects the EMR with the smallest x coordinate, i.e., it places the box as far as possible toward one side of the container. This produces more structured packings along the x axis.

4. **Min Y**
   This heuristic selects the EMR with the smallest y coordinate and therefore prefers placements closer to the back side of the container.

5. **Gravity (Min Z)**
   This heuristic selects the EMR with the smallest z coordinate, which corresponds to placing the box as low as possible in the container.

6. **Anti-Gravity (Max Z)**
   In contrast to the previous heuristic, this strategy selects the EMR with the largest z coordinate and therefore places boxes as high as possible in the container.

7. **Bottom Left Back (BLB)**
   This classical rule selects the EMR with the smallest z coordinate. If several regions have the same value, the smallest y coordinate is selected, and finally the smallest x coordinate is used as a tie-breaking rule.

The set $\mathcal{H}$ is composed of a subset of the implemented heuristics chosen for a given instance, and during the evolutionary process, the algorithm selects which heuristic to apply for each box. If the size of $\mathcal{H}$ is one, i.e., only a single heuristic is available, the heuristic component of the packing rules is not needed, and each box is assigned this heuristic directly.

2.4 The Box Packing Algorithm

Given a list of ordered pending boxes, the algorithm iteratively attempts to place each box into one of the currently opened containers using a placement heuristic. If the box cannot be placed using its current rotation, the algorithm automatically tries all possible rotations. If no valid placement is found after considering all rotations, a new container is opened and the placement is attempted again. The algorithm continues in this way for all boxes. The pseudocode is shown in this section 1.

Algorithm 1 Box Packing Algorithm Pseudocode

```
Input: List of ordered pending boxes B = [b_1, b_2, ..., b_n]
Output: List of opened containers C = [c_1, c_2, ..., c_m]
Initialize empty list of opened containers C ← []
for each box b ∈ B do
    if TryPackBox(b, C) succeeds then
        continue
    end if
    if TryPackWithAnyRotation(b, C) succeeds then
        continue
    end if
    Open a new container c_new
    Add c_new to C
    if TryPackBox(b, C) succeeds then
        continue
    end if
    if TryPackWithAnyRotation(b, C) succeeds then
        continue
    end if
    Error: Box could not be packed, may be too heavy or too large
end for
return C
```

3 Evolution

This chapter introduces an evolutionary approach for improving the quality of packing solutions. Instead of designing a single fixed packing strategy, the algorithm evolves a population of candidate solutions and gradually improves them over time.

The main idea of evolutionary algorithms is to work with a set of candidate solutions, called *individuals*, which are iteratively modified using operators inspired by natural evolution. In each generation, the algorithm evaluates the quality of all individuals, selects the more promising ones, and generates new candidate solutions through various evolutionary operators. Through repeated iterations, the population is gradually refined and the algorithm searches for individuals that produce better packing solutions [6].

3.1 Representation of an Individual

An individual in the evolutionary algorithm represents the packing rules in their vector form, uniquely determining a complete packing solution.

3.2 Fitness Evaluation

The fitness of an individual corresponds directly to the fitness of the packing rules it represents. Individuals representing better packing rules therefore obtain higher fitness values and are more likely to persist in the population in subsequent generations.

3.3 Elitist Genetic Algorithm

The evolutionary process used in this work is based on the elitist genetic algorithm introduced by Gonçalves and Resende [5]. The algorithm parameters are summarized in Table 3.1, and the full pseudocode is given in Figure 2. In each generation, the population is divided into two groups: *elite individuals* and *non-elite individuals*. The elite group consists of the individuals with the highest fitness values, and its size is determined by a predefined parameter.

3.3.1 Selection of Parents

After dividing the population into elite and non-elite individuals, the same number of individuals is randomly selected from both groups. These selected individuals form the parent population used for generating new candidate solutions.

3.3.2 Uniform Crossover

New individuals are generated by applying a *uniform crossover* to pairs of elite and non-elite parents. This type of crossover is called *uniform* because each gene of the offspring is independently chosen from one of the two parents with a

given probability, rather than swapping large contiguous segments of the genome. The probability that a gene is inherited from the elite parent determines how strongly the elite individuals influence the new generation.

3.3.3 New Population Construction

The new population in each generation consists of three parts:

- the elite individuals, which are copied directly to the next generation,
- the offspring generated by crossover,
- a set of newly generated random individuals.

This strategy combines exploitation by preserving the best individuals and exploration by introducing new random individuals. The relative sizes of these groups are determined by the parameters of the algorithm.

3.3.4 Mutation

In addition to the original algorithm, mutation is applied to the offspring generated by crossover. In this approach, each gene in the packing rules vector of an offspring individual is, with a certain probability p_M, replaced by a new random value drawn from its valid range, an interval $[0, 1]$.

The purpose of mutation is to test whether small random modifications can help the algorithm explore new regions of the solution space. In combination with newly generated random individuals, mutation may help discover better packing solutions that would not be reached by crossover alone.

3.3.5 Memetic

Another extension of the original algorithm is the memetic approach. Here, a small number of *hill-climbing* iterations is applied to the mutated individuals.

Hill climbing is a local search that applies small, mutation-like changes with a given probability. For each iteration, a gene is randomly modified and the change is kept only if it improves fitness. This process is repeated for a fixed number of iterations I_{HC}.

We will test whether refining offspring in this way can improve convergence and solution quality by increasing their chances of becoming new elites.

N	population size
G	maximum number of generations
p_E	percentage of elite individuals in the population
p_C	probability that a gene is inherited from the elite parent during uniform crossover
p_M	probability that a gene in a packing rules vector is mutated in offspring
p_R	percentage of newly generated random individuals each generation
I_{HC}	number of hill-climbing iterations applied to each elite individual
p_{HC}	probability that a gene is mutated during each hill-climbing step

Table 3.1 Elitist Genetic Algorithm Hyperparameters

Algorithm 2 Elitist Genetic Algorithm Pseudocode

Input: Parameters N, G, p_E, p_R, p_C, p_M, I_{HC}, p_{HC}
Output: Final population $\mathcal{P}$
$\mathcal{P} \leftarrow$ **RandomPopulation**(N)
Evaluate$(\mathcal{P})$
for $g = 1$ to G **do**
  $N_E = \lceil p_E \cdot N \rceil$ ▷ Determine the number of elite individuals
  $N_R = \lceil p_R \cdot N \rceil$ ▷ Determine the number of random individuals
  $N_S = N - N_E - N_R$ ▷ Determine the number of selected parents
  $\mathcal{E} \leftarrow$ ExtractElite$(\mathcal{P}, N_E)$ ▷ Select the elite individuals
  $\mathcal{N} = \mathcal{P} \setminus \mathcal{E}$ ▷ Obtain the non-elite individuals
  $S_E \leftarrow$ SelectRandom$(\mathcal{E}, N_S)$ ▷ Select elite parents for crossover
  $S_{NE} \leftarrow$ SelectRandom$(\mathcal{N}, N_S)$ ▷ Select non-elite parents for crossover
  $\mathcal{C} \leftarrow$ **Crossover**(S_E, S_{NE}, p_C) ▷ Generate offspring by crossover
  $\mathcal{M} \leftarrow$ **Mutation**$(\mathcal{C}, p_M)$ ▷ Mutate the offspring
  $\mathcal{M} \leftarrow$ **Memetic**$(\mathcal{M}, I_{HC}, p_{HC})$ ▷ Improve mutated individuals by local search
  $\mathcal{R} \leftarrow$ **RandomPopulation**(N_R) ▷ Generate new random individuals
  $\mathcal{P}' \leftarrow \mathcal{E} \cup \mathcal{M} \cup \mathcal{R}$
  Evaluate$(\mathcal{P}')$
  $\mathcal{P} \leftarrow \mathcal{P}'$
end for
return $\mathcal{P}$

4 Experiments

This chapter discusses the results of the experiments conducted for different algorithm settings. We begin by comparing the use of a single default placement heuristic with allowing the evolution to select a specific heuristic for each box. Next, we examine the difference between letting the evolution determine the order of the boxes and using an order based on their volume. After that, we analyze the effect of adding mutation operators and hill-climbing memetic search. Finally, we investigate how assigning weights to the dataset influences the results, comparing cases where the evolution selects the box order with cases where the order is given by volume and, additionally, by the weight of the boxes.

For the elitist algorithm, we primarily adopt the parameter settings proposed by Gonçalves and Resende [5], as shown in Table 4.1, while the remaining parameters are currently set to zero.

TODO: stranka s oznacenim a jejich ustaleni (zadne number of boxes)

N	30× number of boxes
G	300
p_E	10%
p_R	15%
p_C	70%

Table 4.1 Elitist Genetic Algorithm Hyperparameters

4.1 Comparison of Placement Heuristics

This section compares the use of a single default placement heuristic with an approach in which the evolutionary algorithm selects a specific heuristic for each box. Following Gonçalves and Resende [5], we use the Max Distance heuristic as the default one. We then examine whether the results can be improved when evolution is allowed to choose, for each box, from any heuristic listed in Section 2.3.2. In this set of experiments, we consider only datasets without weights.

The results are summarized in Table 4.2. The *All Heuristics* column shows the results obtained when the evolutionary algorithm is allowed to choose the placement heuristic for each box, whereas the *Max Distance* column shows the results obtained when the Max Distance heuristic is used for every box. Each row represents the average fitness of the best solution found for a given dataset type and number of boxes. The results indicate that, in all cases, allowing the evolution to choose from multiple heuristics yields better performance. Based on these results, all subsequent experiments will use the *All Heuristics* setting by default, unless stated otherwise.

TODO: N = pocet boxu

Class	N	All Heuristics	Max Distance
class 1	100	11.918	12.299
class 1	150	18.736	19.283
class 1	50	7.288	7.372
class 2	100	7.884	8.098
class 2	150	11.697	11.933
class 2	50	4.610	4.693
class 3	100	6.174	6.320
class 3	150	9.496	9.756
class 3	50	3.835	3.882
class 4	100	16.097	16.186
class 4	150	24.488	24.912
class 4	50	8.567	8.686

Table 4.2 Comparison of placement heuristics (average best fitness)

4.2 Box Ordering Strategies

This section investigates the impact of box ordering on the performance of the algorithm. We compare a predefined ordering based on box volume with an ordering determined by the evolutionary algorithm. In the volume-based setting, boxes are processed in descending order of their volume, whereas in the evolutionary setting the algorithm is free to choose the order in which boxes are packed.

The results are summarized in Table 4.3. For the larger instances with 150 boxes, the volume-based ordering outperforms the evolutionary ordering on datasets 1 to 3, which are constrained by different size restrictions. In contrast, for dataset 4, where the box dimensions are generated completely at random, the evolutionary ordering performs better in most cases, except for the smallest instances, where the results are comparable. For the medium-sized instances with 100 boxes, the results for datasets 1 to 3 are very similar, suggesting that the choice of ordering has only a minor effect in this setting. For the smaller instances with 50 boxes, the evolutionary ordering yields the best results across datasets 1 to 3.

4.3 Impact of Mutation and Memetic

This section evaluates the effect of incorporating mutation operators and hill-climbing memetic search into the algorithm. To identify suitable parameter settings, we performed a grid search over the memetic and mutation parameters listed in Table 4.4.

The results show that hill-climbing memetic search does not improve the solution quality in practice. In most cases, the results are identical to those obtained without memetic enhancement, while the runtime increases due to the additional evaluation of individuals. For this reason, the hill-climbing component is not recommended and is kept disabled in the remaining experiments.

We also tested memetic search only on elite individuals, instead of applying it

Class	N	Evolutionary Ordering	Volume Ordering
class 1	100	11.918	11.928
class 1	150	18.736	18.705
class 1	50	7.288	7.309
class 2	100	7.884	7.876
class 2	150	11.697	11.650
class 2	50	4.610	4.649
class 3	100	6.174	6.167
class 3	150	9.496	9.483
class 3	50	3.835	3.856
class 4	100	16.097	16.212
class 4	150	24.488	24.524
class 4	50	8.567	8.565

Table 4.3 Comparison of box ordering strategies (average best fitness)

to offspring. In this setting, the results were even worse, since non-elite individuals had fewer opportunities to surpass the elites, which were further improved by memetic search.

Regarding mutation, the setting $p_M = 0.01$ improves the results for the smallest instances with 50 boxes. However, for larger instances, mutation consistently degrades performance. This is expected, since the same mutation probability affects a larger number of genes as the instance size increases, making higher mutation rates overly disruptive in larger problems. Nevertheless, even for lower values $p_M = 0.003$ and $p_M = 0.005$, which should in principle mitigate this effect, no improvement in performance is observed.

The results are summarized in Table 4.5, which reports only the small instances with 50 boxes.

Parameter	Tested values
I_{HC}	1, 2, 5
p_{HC}	0.003, 0.005, 0.01
p_M	0.003, 0.005, 0.01

Table 4.4 Grid search over mutation and memetic parameters

Class	No Mutation	Mutation ($p_M = 0.01$)
class 1	7.288	7.259
class 2	4.610	4.603
class 3	3.835	3.831
class 4	8.567	8.546

Table 4.5 Effect of mutation ($p_M = 0.01$) on small instances (50 boxes)

4.4 Influence of Box Weights

Here, we investigate how assigning weights to boxes affects the results. We compare scenarios where the box order is determined by evolution with those where the order is predefined based on volume and weight.

Conclusion

Bibliography

1. Zizek, Slavoj. *Multiculturalism, or, the cultural logic of multinational capitalism*. New Left Review, 1997.
2. Bernhofen, Daniel M.; El-Sahli, Zouheir; Kneller, Richard. Estimating the effects of the container revolution on world trade. *Journal of International Economics*. 2016, vol. 98, pp. 36–50. issn 0022-1996. Available from doi: https://doi.org/10.1016/j.jinteco.2015.09.001.
3. OByrne, Rob. *Shipping container utilisation: The challenge and why it matters*. 2021. Available also from: https://www.benchmarkingsuccess.com/the-challenge-of-freight-container-utilisation-and-why-it-matters/.
4. Martello, Silvano; Pisinger, David; Vigo, Daniele. The Three-Dimensional Bin Packing Problem. *Operations Research*. 2000, vol. 48, no. 2. Available from doi: 10.1287/opre.48.2.256.12386.
5. Gonçalves, José Fernando; Resende, Mauricio G.C. A biased random key genetic algorithm for 2D and 3D bin packing problems. *International Journal of Production Economics*. 2013, vol. 145, no. 2. issn 0925-5273. Available from doi: https://doi.org/10.1016/j.ijpe.2013.04.019.
6. Eiben, Agoston E.; Smith, J. E. *Introduction to evolutionary computing*. Springer, 2016.
7. Lai, K.K.; Chan, Jimmy W.M. Developing a simulated annealing algorithm for the cutting stock problem. *Computers and Industrial Engineering*. 1997, vol. 32, no. 1. issn 0360-8352. Available from doi: https://doi.org/10.1016/S0360-8352(96)00205-7.

A User Documentation

B Programming Documentation